

LADs: Leveraging LLMs for AI-Driven DevOps

Abstract—Misconfigurations remain a leading cause of failure in cloud deployments, contributing to high operational costs. While Large Language Models (LLMs) have demonstrated promise in generating configuration files, prior approaches largely rely on one-shot prompting and lack the ability to validate or correct erroneous outputs. This paper introduces LADs, an agentic LLM-based framework that automates the complete configuration-to-deployment pipeline through a structured, self-refining loop comprising three core stages: (i) *configuration generation and validation*, where the agent generates config files from the user prompts and retrieved domain knowledge and applies static-schema and unit tests to catch early-stage errors; (ii) *deployment*, where validated configurations are deployed with the resources provided in the user prompts; and (iii) *maintenance*, where the agent analyzes runtime logs and metrics to diagnose failures, generate patches, and reinstate the cycle until a successful deployment is achieved. The core insight is that integrating generation, verification, and feedback within an agentic loop enables automated resolution of configuration errors without human intervention or handcrafted rules. We evaluated across 180 deployment tasks spanning Dask, Redis, and Ray, LADs improves deployment success rates from $\leq 22\%$ with simple prompting to 68%, corrects 94% of runtime failures within three iterations, and generates production-ready configurations in 4–11 seconds at a cost four orders of magnitude lower per invocation compared to a conventional developer. These results highlight LADs as a scalable and cost-efficient solution for end-to-end DevOps automation of distributed systems in cloud-native environments.

Index Terms—Automated DevOps, Large Language Models, Agentic AI Systems, Distributed Systems

I. INTRODUCTION

Cloud-based systems involve complex deployment and continuous maintenance, which require comprehensive expertise in software development, systems engineering, security, networking, data, operations management, and domain-specific knowledge [1]. This complexity and the continuous need for configuration updates can be time-consuming and expensive for companies that rely on computing, storage, data processing, and ML systems and have limited resources [2], [3].

At large-scale companies like Meta, thousands of configuration file changes are committed daily. These changes outpace code updates and misconfigurations remain a top cause of production incidents [4]. The rapid pace of configuration changes in modern distributed systems applications shows a pressing need for automated intelligent solutions that can handle dynamic environments with minimal human intervention. These rapid changes are fueled by continuous deployment of new features, autoscaling responses to volatile traffic patterns, and the constant adjustment of security or compliance policies [5].

A recent study indicates that 77% of organizations rely on DevOps to deploy their services [6]. Yet many struggle with effective management because DevOps teams spend significant

time on observability, monitoring, and security tasks, which consume 54% of their efforts. Only three out of ten organizations fully understand their cloud spendings [6], highlighting resource-budgeting challenges that grow more complex in heterogeneous architectures hosting distributed services for data processing, caching, and ML workloads. These budgeting challenges are often amplified by misconfigurations or suboptimal system settings, which can result in inefficient resource utilization and inflated cloud costs.

Existing solutions often remain system-specific and fail to generalize across different services, leading to extensive manual tuning and frequent misconfigurations. For example, tuning an Log-Search Merge-Tree (LSM)-based key-value store like RocksDB [7] involves over 100 parameters [8], [9], while large-scale platforms like Spark [10] and Dask require careful optimization of parameters such as `spark.executor.memory` and `spark.core.max` to meet performance goals [11]. Engineers often build validators for these large parameter spaces, yet many remain uncovered due to cost and complexity [4]. Such issues are compounded by dynamic workloads, evolving user goals, and the need for real-time reconfiguration to maintain reliability and efficiency. Automated monitoring and maintenance are also essential to avoid failures caused by version mismatches, poor memory allocation or misconfigured network policies [12]. DevOps engineers often rely on tools like Prometheus [13] for observability, yet these tasks demand significant time and are complicated by limited visibility into operational costs. Since, each tweak in the configurations requires human effort and every misconfiguration directly costs CPU/GPU-hours, memory-hours, etc and leads to extra cost.

Recent advances in Large Language Models (LLMs) suggest a path toward streamlining these workflows. However, turning LLMs into reliable “autonomous DevOps” systems is non-trivial: effective solutions must integrate domain-specific knowledge, generate correct configurations under constraints, debug and repair failures robustly, and adapt across heterogeneous services and environments. In this paper, we present LADs, an agentic LLM system for fully automating the configuration, deployment, and maintenance of distributed cloud services. Unlike prior LLM-based DevOps tools that address configuration generation, validation, or debugging in isolation [14]–[16], LADs is the first system to unify these stages into a single closed-loop pipeline that executes, observes, and repairs real cloud deployments without human intervention.

The central insight behind LADs is that configuration correctness cannot be established at generation time alone: it must be validated through execution. By embedding LLM-driven reasoning inside a closed feedback loop that couples

configuration synthesis with live deployment and runtime signals, LADs enables autonomous correction of misconfigurations that static or one-shot approaches cannot resolve.

LADs operates as a modular agentic system spanning the DevOps lifecycle. Retrieval agents first distill task-relevant context from user intent, documentation, prior configurations, and cloud constraints. Configuration-generation agents then synthesize executable Infrastructure-as-Code artifacts (e.g., Kubernetes YAML and Helm charts) while enforcing schema and unit-level validity checks. Deployment agents apply these configurations using standard cloud tooling, and monitoring agents continuously analyze logs and metrics from production observability stacks. When failures or violations occur, repair agents diagnose root causes and generate targeted configuration patches, which are re-deployed in subsequent iterations until the system converges to a stable, policy-compliant state.

This closed-loop design elevates LLMs from passive code generators to active system controllers. Rather than relying on handcrafted validators or system-specific heuristics, LADs leverages runtime feedback to iteratively refine deployments, allowing it to generalize across heterogeneous services and dynamically changing workloads. We demonstrate this capability on three widely used distributed platforms: Dask, Redis, and Ray, each with large and complex configuration surfaces.

Our key contributions and findings are as follows:

- **End-to-end autonomous DevOps system.** We design and implement LADs, the first agentic LLM system that unifies configuration generation, deployment, monitoring, and repair into a single closed feedback loop for distributed cloud services.
- **Cost-aware agent orchestration.** We introduce feedback-driven prompt chaining combined with retrieval and few-shot context, reducing tokens by $2.1\times$ while improving deployment success from 22% to 68%.
- **Runtime-aware self-repair.** We show that integrating live execution signals enables LADs to autonomously resolve 94% of configuration-induced runtime failures without human intervention.
- **Benchmark for autonomous DevOps.** We release a 180-task benchmark spanning Dask, Redis, and Ray, with both static and dynamic validators, enabling reproducible evaluation of end-to-end DevOps automation systems.

II. RELATED WORK

While LLMs have shown promise across individual configuration-related tasks, such as generating deployment files, tuning hyperparameters, or debugging errors, most existing approaches focus on these problems in isolation, lacking an adaptive, end-to-end framework that unifies them. Kannan et al. [17] employ LLMs for software tool configuration, particularly hyperparameter tuning in ML applications. Similarly, PreConfig [15] applies text-to-text transformations for network configuration but lacks adaptability to dynamic cloud environments. Mondal et al. [18] demonstrate the potential of LLMs in router configuration but highlight their susceptibility to critical errors, necessitating manual validation. Unlike these

methods, LADs unifies configuration synthesis, deployment, and failure recovery into a single agentic loop. It closes the feedback cycle by continuously refining configurations based on static analysis and runtime signals, enabling real-time error correction and autonomous convergence, capabilities that are not supported in prior methods.

LLMs have also been explored for debugging and validation. Fu et al. [16] and Lian et al. [5] use LLMs to identify misconfigurations in distributed systems, but their approaches are reactive, focusing on error detection post-deployment rather than preventing misconfigurations from occurring. Similarly, IaC-Eval [14] introduces a benchmark for LLM-generated Infrastructure-as-Code (IaC), revealing challenges in static correctness evaluation but lacking mechanisms for adaptive tuning. LADs differs by taking a proactive approach, leveraging Retrieval-Augmented Generation (RAG), Feedback-Based Prompt Chaining, and structured logs to iteratively refine configurations before deployment, reducing the likelihood of failures rather than just identifying them.

CloudEval-YAML [19] is another recent work like IaC-Eval [14] that provides executable unit tests for cloud configurations using Minikube-based Kubernetes clusters, yet remains scoped to controlled test environments without measuring real-world performance, cost, or resource consumption under actual workloads. AIOpsLab [20] offers a framework to assess AI agents in operational settings with real deployments and injected failures. These establish important evaluation methodologies but serve primarily as testbeds rather than complete deployment synthesis and repair systems.

Beyond configuration validation, Ruiz [21] and Malul [22] propose post-deployment strategies for automated software improvement and misconfiguration detection, respectively. While these methods enhance system robustness, they primarily respond to issues rather than optimizing configurations dynamically. In contrast, LADs integrates Few-Shot Learning, Chain-of-Thought reasoning, RAG, and structured feedback loops in a hybrid unified mechanism to continuously refine and optimize distributed systems configurations throughout the deployment lifecycle, balancing performance, cost, and reliability. Moreover, LADs takes into account system and network resource boundaries and ensures configuration validation through execution.

Metric	Redis	Ray	Dask
Max Keys	221	110	192
Max Depth	7	8	7
Avg Keys	77 $\pm$ 44	44 $\pm$ 20	82 $\pm$ 35
Avg Depth	5.0 $\pm$ 0.52	5.5 $\pm$ 0.68	5.1 $\pm$ 0.50
No. of Features	19	15	16

TABLE I: Comparison of Configuration Complexity based on number of configurable parameter keys, depth of nested parameters, and features tested.

III. DESIGN

A. Research Questions

To design LADs, we address three core research questions that guide our architectural decisions.

RQ1: Structural Correctness. How can we ensure LLM-generated configurations are valid syntactically and correctly execute the complex nested YAML schemas required by distributed systems like Dask, Redis, and Ray?

RQ2: Context Integration. How can we effectively and efficiently provide contextual information from diverse sources such as configuration schemas, application documentation, user intent, and resource constraints, to guide accurate configuration generation across heterogeneous systems?

RQ3: Autonomous Repair. How can we automatically detect deployment failures and repair the relevant configurations through continuous monitoring and iterative feedback without any additional human intervention?

To answer these questions, we use an iterative design methodology: we first evaluate individual prompt engineering techniques (instruction prompting, RAG, few-shot learning, chain-of-thought) on a human-curated dataset through static validation, then we synthesize the most effective techniques for use in a unified multi-agent architecture.

B. Benchmark and Dataset:

To thoroughly and efficiently evaluate our design decisions for automated configurations and deployment of distributed systems applications with LADs, it is necessary to have a benchmark dataset. The static validation checks whether the generated configuration aligns with the user intent, similar to IaC-Eval [14].

However, to the best of our knowledge, no public datasets exist that validate user intent, deployment success, and performance for distributed systems application configuration. *To address this gap, we created a human-curated dataset that we plan to release publicly.* This dataset tests configuration generation for distributed systems like Dask [23], Redis [24], and Ray [25], and we continue to add more systems. We chose these systems because of their popularity (based on GitHub stars) and because they represent different microservice tasks, including computing, caching, storage, and ML. They also offer varying configuration complexity levels, as shown in Table I. The Table compares the configuration complexity of Redis, Ray, and Dask based on the max and average number of configurable key parameters, the max and average depth of nested configurable parameters, and the number of features that were evaluated for each distributed system in our dataset. Redis has the highest Max Keys (221) and Avg Keys (77 ± 44), indicating its vast configuration space, and it is also the most rigorously validated with 19 features in our dataset, while Ray has the deepest structure (Max Depth = 8, Avg Depth = 5.5 ± 0.68), suggesting more hierarchical complexity. Hierarchical complexity refers to the depth and fan-out of nested configuration objects, for example, a Helm chart that embeds sub-charts, each defining its own Deployment, Service, and HPA resources; we quantify this as the

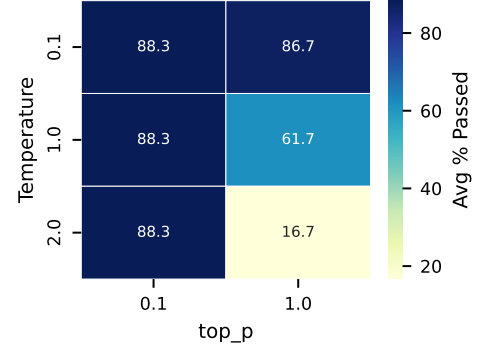


Fig. 1: Average Performance across all difficulty levels with varying Temperature and top_p parameters.

average YAML nesting depth plus the count of cross-resource references (e.g., serviceAccountName, configMapRef). Dask offers a balanced profile (192 keys, 16 features), making it relatively easy to configure.

The dataset itself includes user intents, prompts, and validators for each distributed service, with 60 data points per service. These 60 configurations per service are categorized into six difficulty levels, ranging from beginner-friendly tasks (Level 1) to expert-level challenges (Level 6). The prompts in the dataset define the high-level task, such as: “Create a basic Dask cluster for deployment on Kubernetes in the namespace ‘dask,’ with scheduler name ‘dask-scheduler.’” Whereas, the user intents specify precise requirements, e.g., “The Dask scheduler should include one scheduler and two worker nodes.” During evaluation with this benchmark, the LLM agent takes the user intent and prompt as input and generates an output configuration. The Python-based validator functions provide a fine-grained method to verify whether the generated configuration aligns with the prompt and the user intent. Unlike IaC-Eval [14], which relies on regex-based checks, we implement our validators in Python because it offers greater flexibility and aligns better with LLM capabilities [26]–[28]. Recent work shows that LLMs are particularly effective at generating Python-based verifiers [26], [29], making our choice well-suited for future automation of validator generation by LLM agents. Two independent annotators assigned each task to one of six difficulty levels based on number of configurable keys, nesting depth, and cross-resource dependencies. Tasks were randomly sampled from public GitHub repositories.

C. Design decisions

1) *Structural integrity of configurations:* The non-deterministic nature of generative models [30] means a single prompt can generate a different response each time, which complicates the consistent extraction of the information needed to form a syntactically correct configuration file. This requires controlling model randomness and cleaning the generated responses.

a) *Pre-processing Inference:* We supply the LLM with a sample YAML template to guide its output format. We parse the response and strip any segments that deviate from the

template, removing explanations, reasoning or examples that would break the YAML. We then validate the cleaned output against the schema. Once the result conforms to valid YAML, we adjust the model’s randomness settings.

b) *Controlling model randomness*: We adjust two key parameters, *temperature* and *top_p*, to balance diversity and fidelity. Temperature scales logits before softmax. Low values bias toward high probability tokens for deterministic outputs. High values increase variability. *top_p* retains the smallest token set whose cumulative probability exceeds *p*, filtering unlikely tokens and preserving variety. Figure 1 shows that higher settings degrade configuration quality and determinism. Based on an ablation study on a validation set curated manually, we set both parameters below 0.1 to ensure stable, accurate configuration generation.

c) *Instruction Prompting (IP)*: Apart from parameter optimization, instruction prompting [31] is vital for maintaining the structural integrity of generated YAML files. Figure 2 illustrates an engineered prompt that instructs the LLM to produce only relevant configuration details. Without IP, as shown in the figure, we can observe severe inconsistencies that render the YAML files unusable. We use IP, data pre-processing, and randomness control to avoid generating syntactically incorrect configurations. While these basic techniques help the models pass some prompt-alignment checks, the performance remains insufficient for fully automated DevOps pipelines.

2) *Context Provision*: To address deployment complexity under varying resource conditions, workload dynamics, or changing user intent, configuration generation requires contextual data that includes system information, resource limits, application documentation, and user intent. We categorize this context as either *structured* (e.g., reference outputs and configuration examples) or *unstructured* (e.g., application documentation, user intent, and system data). Common methods for incorporating context include RAG [32], few-shot [33], [34], and chain of thought [35].

a) *Retrieval Augmented Generation (RAG)*: We first evaluate the RAG optimization by providing structured and unstructured information via a stuff chain, FAISS retriever [36], a 5000-token chunk size, and a 500-token overlap, which is a standard configuration for FAISS. Table II compares this approach to the instruction prompting optimization. The performance gains are minimal because structured schema information loses coherence when broken into chunks, and there is a risk of introducing fragmented or noisy content [37]. Moreover, RAG does not guarantee the retrieval of essential chunks, so if the retrieved chunk contains the configuration schema, critical configuration formatting details may be lost. This explains why we see only modest improvements over IP alone. These findings lead us to a key observation: **OB① RAG is better suited for passing unstructured context information that can be filtered and utilized in the form of relevant chunks of information required. This reduces the amount of data passed via input saving valuable input tokens for other structured information.** To resolve this issue, we next consider passing structured configuration

Engineered Prompt: You are a highly skilled DevOps engineer specializing in Dask configurations and are tasked with generating YAML configurations. Generate a Redis configuration in valid YAML format based on the following requirements: Prompt: {original prompt} User Intent: {intent} Output only the configuration in valid YAML format. Do not include any comments, explanations, or additional content.

Without Prompt Engineering

```
1 ##### 1. Add the Dask Helm Chart
   Repository
2 First, add the Dask Helm chart repository
   if you haven't:
3 helm repo add dask https://dask.github.io
   /helm-chart
4 helm repo update
5
6 ##### 2. Create a Namespace for Dask
7 kubectl create namespace dask
8
9 ##### 3. Configure Helm Values
10 scheduler:
11   replicas: 1
12 worker:
13   replicas: 10
14 daskGateway:
15   enabled: false
16 ...
17 ##### 4. Deploy Dask Using Helm
18
```

Prompt Engineered

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: dask-worker-deployment
5   namespace: dask
6 spec:
7   containers:
8     - name: dask-worker
9       image: daskdev/dask:latest
10     resources:
11       requests:
12         cpu: "1"
13         memory: "2Gi"
14       limits:
15         cpu: "1"
16         memory: "2Gi"
17 ...
18
```

Fig. 2: Comparison of LADs output without vs. with prompt engineering.

schema context via a more direct approach through the few-shot technique.

b) *Few-Shot Learning (FSL)*: For the few-shot technique, we draw on prior work in structured code generation [5], [14], which demonstrates that providing example shots significantly improves performance. The LLM agent benefits from having

Few Shot-learning

```

1 You are a highly skilled DevOps engineer
  specializing in Dask deployment and
  are tasked with generating YAML
  configurations.
2
3 Here are a few examples that you can learn
  from:
4
5 Example 1:
6 {
7   /* Example1 engineered prompt */
8   "config": {
9     /* Correct Configuration response: */
10    apiVersion: autoscaling/v2beta2
11    kind: HorizontalPodAutoscaler
12    metadata:
13      ...
14  },
15 },
16 Example 2:
17 ...
18
19 Prompt: {engineered prompt}
20 User Intent: {intent}
21

```

Fig. 3: Example usage of the few-shot technique for configuration generation in LADs.

a clear reference for both the schema and the type of content expected. We supply in-context few-shot examples of prompts, user intents, and both correct and incorrect outputs, as illustrated in Figure 3. Table II confirms that few-shot examples lead to substantial gains in accuracy. This success is driven by the LLM’s strong in-context learning ability, which helps it better interpret configuration requirements and user intents. We use three prompts, we developed three prompts for each of the three systems (Dask, Redis, and Ray), representing different levels of difficulties.

c) *Chain-of-Thought (CoT)*: For chain-of-thought optimization, we adopt the zero-shot learning method from related work [35]. This method prompts the LLM to reason in steps by inserting instructions such as “Let’s think step by step,” after each response. Table II shows only modest improvements with this approach. The limited gains arise because the LLM lacks sufficient domain-specific knowledge (e.g., system specifications and configuration schema) to leverage multi-step reasoning for configuration generation.

By contrast, the most significant improvements come from providing example shots as input, giving the model a structural reference for valid configurations. These findings leads us to a key observation: **OB② LLMs need a more concrete schema or format to guide its outputs. Few-Shot and CoT methods are better suited for passing structured information. Few-Shot examples help models learn the configuration format and structure, while CoT enables them to correct issues like overlooked user intent.**

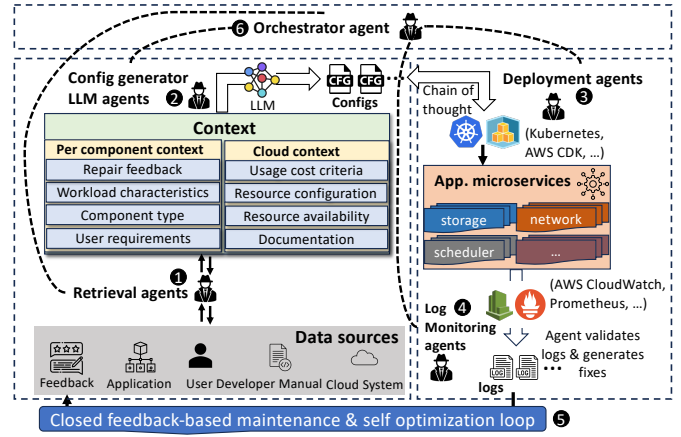


Fig. 4: High-level overview of LADs for automated distributed systems application management. The system leverages LLM agents and integrates in-context few-shot and instruction prompting with RAG and prompt chaining.

D. LADs

After validating various design optimizations, we unify the best techniques into LADs, consisting of six specialized agents organized in a closed feedback loop as shown in Figure 4.

1) *Optimization Integrations*: Following our design analysis, we integrate a hybrid optimization strategy into LADs to address the distinct challenges of configuration generation and maintenance. We employ Instruction Prompting (IP) to enforce strict syntactic integrity and Few-Shot Learning (FSL) to ground the model in correct schema structures, as our evaluation showed FSL outperforms RAG for structured data generation. Retrieval Augmented Generation (RAG) is therefore reserved for ingesting unstructured context like documentation, while Chain-of-Thought (CoT) reasoning is applied specifically within the feedback loop to diagnose failures. These techniques are unified through Feedback-Based Prompt Chaining, enabling the system to iteratively refine configurations based on runtime signals.

2) *Agentic architecture*:

① **Retrieval Agent**. DevOps engineers normally wade through heterogeneous artifacts, cluster state dumps, prior deployment manifests, application requirements, cost policies, and failure logs, most of which are irrelevant for the current intent. The retrieval agent replaces this manual triage: using its tool palette (file/directory browser, semantic search over docs & logs, cost-metadata fetcher, and lightweight schema summarizer) it extracts only task-critical facts (e.g., required ports, persistence mode, resource ceilings). It emits a structured context bundle that becomes the grounded prompt for the next agent.

② **Configuration Generation Agent**. Given the curated context, the config generation agent’s purpose is to translate intent + environment facts into executable Kubernetes/Helm (and service-specific) manifests while pre-empting common misconfigurations. It leverages tools for template generation

and policy/schema validation. Its output is a validated manifest set or a self-critique that triggers another internal draft. The output from this agent is a final configuration that is ready to be deployed by the next agent.

③ **Deployment Agent.** The deployment agent operationalizes approved manifests, eliminating brittle human hand-offs. Using IaC and orchestration tools like kubectl/Helm runner and Kubernetes, it applies resources, instruments them with labels for cost/SLO tracking, and returns deployment receipts (object statuses, endpoints, and any immediate warnings) for downstream monitoring.

④ **Monitoring & Feedback Agent.** Once services are live, the monitoring agent’s purpose is continuous assurance against drift and runtime failures. Through its tools, metrics collector & log stream collector, and event parser it filters raw telemetry into concise “issue capsules” (symptom signature + localized suspect fields). These capsules feed the repair workflow without overwhelming the agent with noise.

⑤ **Prompt Chaining & Repair Orchestrator.** Cross-cutting the loop, the prompt chaining module coordinates iterative improvement, assembles retrieval context and current failure capsule + last manifest diff, and applies reasoning sub-prompts (root-cause hypothesis to minimal patch plan to revised manifest). Its role is to ensure each cycle is targeted (change only what evidence implicates) and data-efficient, converging quickly toward a stable, policy-aligned deployment.

⑥ **Orchestrator agent.** The orchestrator receives requests and manages communication and execution order for independent agents, providing the global context necessary to prevent infinite loops.

IV. IMPLEMENTATION AND ADOPTION

a) *Implementation:* We implement LADs using n8n [38] for agent development. The backend server is deployed on AWS EC2 [39]. The data, such as user credentials and API endpoints, are stored in AWS S3 [40] in an encrypted format. LADs also includes a GUI deployed on AWS EC2. The setup is highly modular and flexible. New agents and tools used by agents can be added simply through drag-and-drop in the n8n workflow. The backend is also modular, and new routes can be integrated by registering new user-defined functions. **Due to its cloud-agnostic design and reliance on standard containerized and API-driven components, LADs can be easily ported to other cloud providers such as Microsoft Azure, Google Cloud Platform (GCP), or on-premise Kubernetes-based environments with minimal changes.** The users can either provide ssh credentials to their local cloud endpoint running Kubernetes or AWS credentials through the GUI using natural language, and the credentials will be filtered out and stored in the AWS S3 DB. From there, the n8n workflow can fetch the credentials and get the status of the local or AWS cloud services of the user to gain context. Once the context is established, the user can send queries via the same GUI, which will be routed through the backend to the orchestrator agent in n8n. The agents in n8n have the ability to generate

Models	IP	CoT	RAG	FSL	LADs
Dask					
CL-7B	5.00	0.00 ↓	6.67 ↑	28.34 ↑	33.34 ↑
CL-13B	5.00	3.34 ↓	8.34 ↑	43.34 ↑	48.34 ↑
L3.1-8B	18.34	8.34 ↓	18.34 ↑	46.67 ↑	40.00 ↑
DS2-16B	8.34	20.00 ↑	10.00 ↑	48.34 ↑	50.00 ↑
Q2.5-7B	21.67	16.67 ↓	15.00 ↓	55.00 ↑	56.67 ↑
Q2.5-14B	48.34	45.00 ↓	18.34 ↓	56.67 ↑	68.34 ↑
Redis					
CL-7B	16.67	1.67 ↓	16.67 ↑	26.67 ↑	28.34 ↑
CL-13B	15.00	1.67 ↓	13.33 ↓	25.00 ↑	23.34 ↑
L3.1-8B	21.67	23.34 ↑	21.67 ↑	35.00 ↑	36.67 ↑
DS2-16B	20.00	25.00 ↑	23.34 ↑	30.00 ↑	38.34 ↑
Q2.5-7B	20.00	20.00 ↑	16.67 ↓	30.00 ↑	31.67 ↑
Q2.5-14B	45.00	43.34 ↓	31.67 ↓	31.67 ↓	58.34 ↑
Ray					
CL-7B	3.34	0.00 ↓	1.67 ↓	35.00 ↑	41.67 ↑
CL-13B	0.00	1.67 ↑	0.00 ↑	35.00 ↑	41.67 ↑
L3.1-8B	3.34	3.34 ↑	3.34 ↑	51.67 ↑	56.67 ↑
DS2-16B	5.00	5.00 ↑	13.34 ↑	28.34 ↑	30.00 ↑
Q2.5-7B	20.00	20.00 ↑	18.33 ↓	61.67 ↑	53.34 ↑
Q2.5-14B	30.00	26.67 ↓	18.34 ↓	63.34 ↑	70.00 ↑

TABLE II: **Performance of Various LLMs on Multiple Applications with Different Optimizations.** The highest-performing optimization for each LLM is bolded, while the overall best-performing pair is bolded and underlined. Abbreviations: **CL-7B** (CodeLlama7b), **CL-13B** (CodeLlama13b), **L3.1-8B** (Llama 3.1 8b), **DS2-16B** (DeepSeekV2 16b), **Q2.5-7B** (Qwen2.5 7b), **Q2.5-14B** (Qwen2.5 14b). Arrows indicate change from IP: green (↑) for improvements and red (↓) for decreases.

configuration, deploy, and monitor distributed systems on the provided cloud endpoint, as shown in Figure 4.

b) *Adoption:* LADs is deployed and fully functional and is currently being used by research labs across universities to manage their distributed systems services for research purposes. It is also being piloted across several different industrial partners for feedback and improvement. We also plan to make LADs open-source for research and improvement purposes.

V. EVALUATION

A. Experimental Setup

a) *Static and Dynamic Eval:* We use a two-fold approach for evaluation using static and dynamic validation techniques borrowed from Software Engineering [41]. Our static validation benchmark consists of 180 data samples spread across 3 different difficulty levels of distributed systems. For each distributed system we have 6 different difficulty levels with 10 data samples per difficulty level, forming 60 data samples per distributed system as explained in detail in Section III.

Since correctness cannot be established at generation time alone, we also validate our unified 6-agent architecture through execution, using dynamic validation. *For the dynamic validation, to ensure a fair evaluation, we do not use the dataset utilized for static validation because that dataset is used to inform our design choices.* Instead, we deploy the system in multi-tenant cloud environments and assess its performance

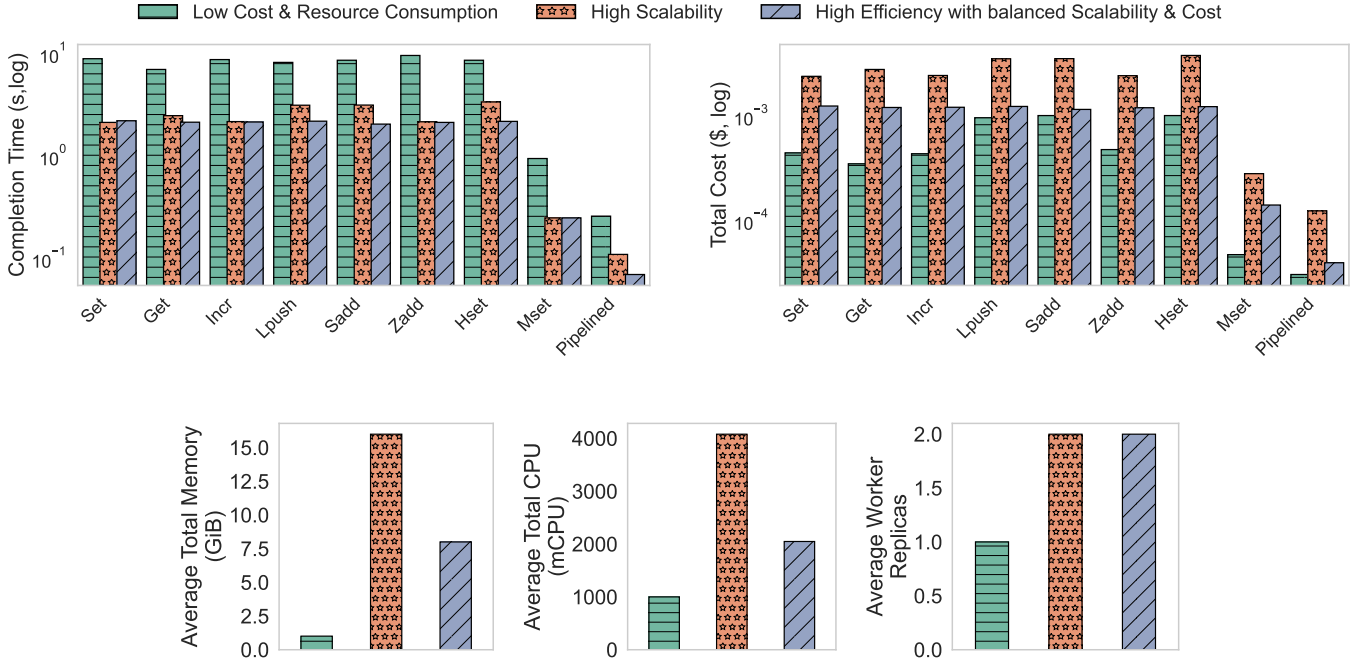


Fig. 5: Dynamic validation of Redis over 9 different benchmarks for different User Intents. Log-scale Completion times (top left) and Benchmark Processing Costs (top right) of benchmarks, allocations of Memory (bottom left), CPU as milliCPU, or 1/1000 of a CPU (bottom mid), and replicas (bottom right).

through official benchmark applications from each system’s documentation [42]–[44]. We use a containerized setup with Kubernetes, commonly employed for distributed microservices like Dask, Redis, and Ray. We define three user intents widely used in related DevOps studies [45]: “Low cost and resource consumption,” “High scalability,” and “High efficiency balancing processing costs, resource consumption, and scalability.” The first two focus on simpler metrics such as latency, cost, and resource usage, while the third presents a complex trade-off scenario. These intents are provided to the LLM agent, which generates a configuration and deploys the system on Kubernetes. We measure alignment with the user intent by running the official benchmarks while tracking completion time, cost, and resource consumption. The deployment is continuously monitored to validate if the configuration meets the user’s performance criteria. We measure the performance on 5 axes, namely Completion Time, Total Cost, Average Total Memory, Average Total CPU, and Average Worker Replicas.

b) Models and Libraries: We use several different *coding specialized LLMs* for evaluation for static validation, including Qwen Coder 2.5-7B and Qwen Coder 2.5-14B [46], DeepSeek Coder V2-16B [47], CodeLlama-7B and CodeLlama-13B [48], and Llama 3.1-8B [49]. The LLM agents are created using n8n [38], HuggingFace [50], and Ollama [51] libraries in Python. For dynamic validation, we choose the best-performing model from these due to the steep cost overhead of dynamic validation-based evaluation. Notably, as per our knowledge, ours is the first work that does dynamic validation

while related works only rely on static validation [14], [15].

B. Results and Discussion

1) Static Validation: In this section, we evaluate the accuracy of different LLMs in generating valid configurations through static validation. The purpose of this experiment is to assess whether LLM-generated configurations align with user intent without requiring actual deployment. Table II summarizes the results across three distributed systems: Dask, Redis, and Ray highlighting the best-performing optimization strategy for each model.

OB③ Among optimization techniques, FSL emerged as the most effective in improving configuration accuracy. This aligns with findings from prior work, which highlight FSL’s ability to enhance structured generation by providing in-context learning examples [5], [14]. The insights from our results in Table II highlight an observation, **OB④ CoT used to enhance advanced reasoning abilities, did not result in noticeable improvements over vanilla models.** However, this technique can be useful to perform validation of configuration when static human-curated validators are not available which is why we have used it in LADs for this purpose.

OB⑤ Relatively recent models despite their smaller size perform better than the larger but older models. For example, Table II shows Llama 3.1 (8B) and Qwen 2.5 (7B) both outperform CodeLlama (13B) and DeepSeek V2 (16B). This indicates that configuration generation tasks can be handled effectively by smaller, computationally efficient models

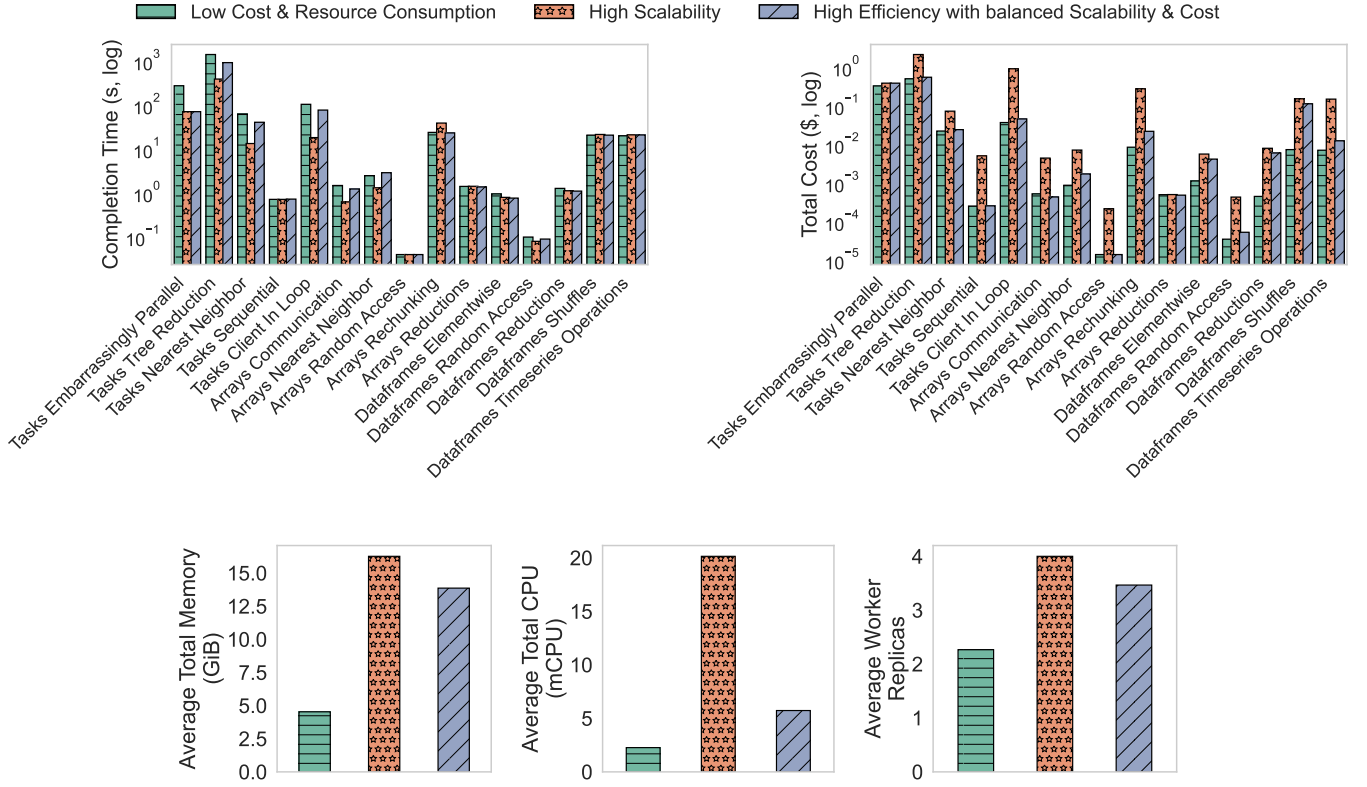


Fig. 6: Dynamic validation of Dask over 15 different benchmarks for different User Intents. Log-scale Completion times (top left) and Benchmark Processing Costs (top right) of benchmarks, allocations of Memory (bottom left), CPU as milliCPU, or 1/1000 of a CPU (bottom mid), and replicas (bottom right).

rather than expensive large models. This discrepancy can be attributed to the newer architectures and improved training datasets of Llama 3.1 and Qwen2.5, compared to the older, fine-tuned CodeLlama models [52]. Notably, CodeLlama is based on Llama 2, which is nearly two years old, suggesting that newer models benefit from higher-quality training data. In high-frequency API calling scenarios, this independence from large models is particularly beneficial, as it enables faster, more cost-effective automation without sacrificing accuracy. Analyzing model performance across different LLMs, Qwen2.5 14B was the overall best performer, consistently achieving the highest accuracy across multiple applications. Given its strong results, we select Qwen2.5 14B as the primary model for conducting all subsequent dynamic validations.

2) Dynamic Validation:

a) *Redis*: The dynamic validation results in Figure 5 show the completion times and the processing costs of benchmarks for different benchmark applications run on Redis deployment via LADs. The Completion times and corresponding cost results reflect that LADs is taking user intent into account as “Low Cost and Resource consumption” leads to higher completion times due to less amount of resources allocated and this also leads to lower costs. Similarly, “High scalability” results in lesser completion time due to the abundant allocation

of resources for executing the benchmark application but results in a higher cost. The more complex case of maintaining “High Efficiency with balanced Scalability and Cost” results in the lowest completion time for most benchmark applications and also balances between costs and resource consumption, costing lower than the “High Scalability” intent but higher than “Low Cost and Resource consumption”.

We observe a trend in memory, CPU, and replica resource consumption as shown in Figure 5. The “Low Cost and Resource consumption” intent shows the lowest allocation, followed by “High Efficiency with balanced Scalability and Cost,” and finally “High scalability” exhibits the highest allocation. *These results demonstrate that LADs can serve as a one-stop solution for accurate configuration generation and deployment of various distributed systems, aligning with both simple and complex user intents and dynamic, heterogeneous system resources.*

We also extend this analysis to other distributed systems like Dask and Ray across multiple benchmarks. We show the results of Redis in Table II in section §V-B2. The results provide insights into the system’s adaptability, resource allocation, and cost efficiency under varying workloads.

b) *Dask*: Figure 6 presents the performance of Dask across 15 different benchmarks. The top-left subfigure shows

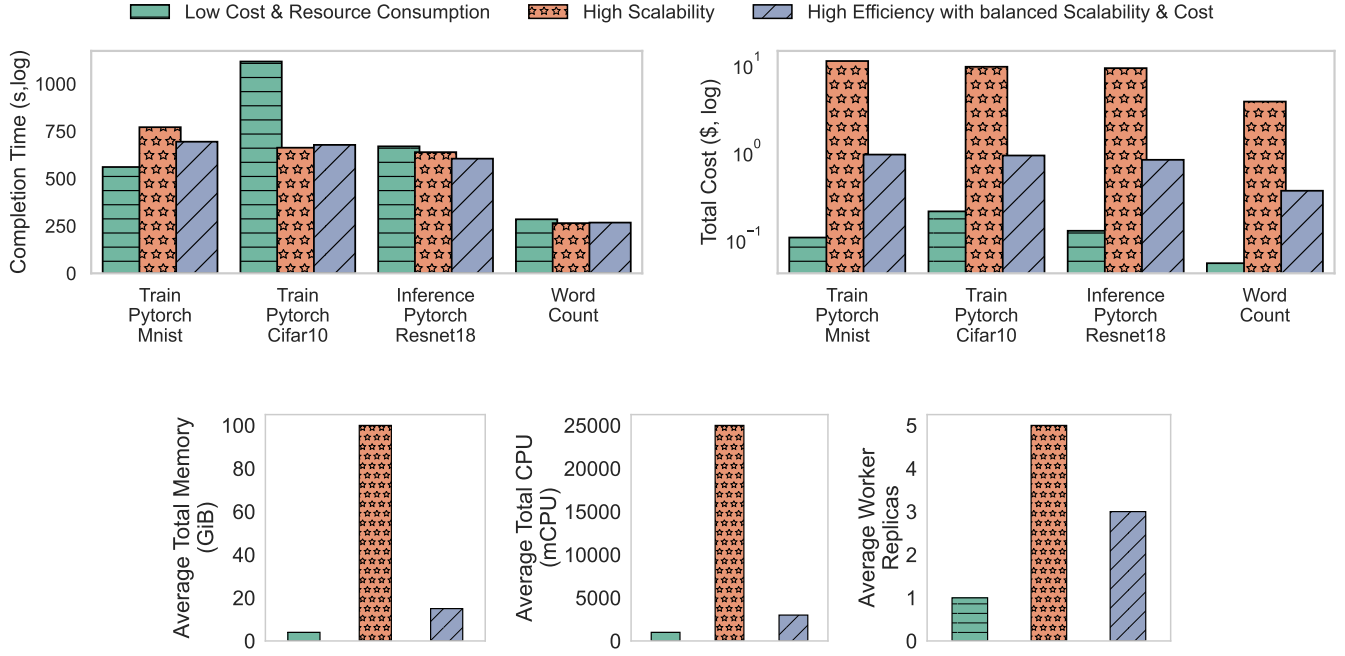


Fig. 7: Dynamic validation of Ray over 4 different benchmarks for different User Intents. Log-scale Completion times (top left) and Benchmark Processing Costs (top right) of benchmarks, allocations of Memory (bottom left), CPU as milliCPU, or 1/1000 of a CPU (bottom mid), and replicas (bottom right).

the log-scale completion times, highlighting the variations in execution efficiency. The top-right subfigure illustrates the benchmark processing costs, reflecting the economic impact of different configurations. The bottom row presents resource allocation trends, including memory requests (bottom left), CPU allocation (bottom middle), and worker replicas (bottom right). The results demonstrate that LADs effectively aligns the configuration according to user intent. Some Dask benchmarks such as “Tasks Nearest Neighbor” and “Tasks Client In Loop” are more efficient with “High Scalability” compared to the “High Efficiency with balanced Scalability & Cost” because the latter not only prioritizes efficiency but also balances scalability and costs. This shows that LADs can successfully handle complex user intents that involve various resource and performance tradeoffs.

c) *Ray*: Figure 7 showcases the dynamic validation of Ray over four different benchmarks. The top-left subfigure compares log-scale completion times, while the top-right subfigure highlights the benchmark processing costs. The bottom row further breaks down the memory (bottom left), CPU (bottom middle), and replica (bottom right) allocations. The findings indicate that LADs successfully enhances resource utilization and cost-effectiveness, making Ray more adaptive to workload variations.

d) *Feedback-based prompt chaining*: Figure 8 shows how feedback-based prompt chaining enables the resolution of errors and Warnings to get accurate configurations. **OB@ Approximately 94% of configuration errors can be re-**

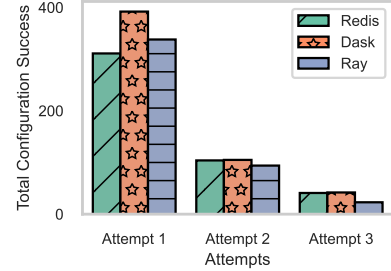


Fig. 8: Feedback-based prompt chaining

Systems	Avg. Latency (s)	Std. Dev. (s)
Dask	9.282	0.849
Redis	4.772	0.462
Ray	10.524	0.159

TABLE III: Overhead of Configuration Generation by via LADs per API call.

solved through feedback-based prompt chaining technique in LADs by the third attempt.

C. Overhead comparison with LADs

As shown in Table III LADs takes between 4 to 11 seconds on average for configuration generation using LLM agents. This process is significantly more efficient than manually

System	Output Tokens	Input w/ RAG	Input w/o RAG	Cost w/ RAG (\$)	Cost w/o RAG (\$)
Dask	67	1200	7100	0.00024	0.00142
Redis	183	1500	3000	0.00030	0.00060
Ray	72	1320	2800	0.000264	0.00056

TABLE IV: Input/output tokens and total cost per configuration for distributed systems with and without RAG.

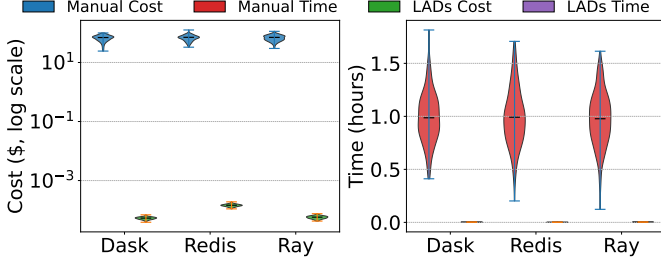


Fig. 9: Violin plots of manual (developer) vs. LADs deployment cost and time distributions for Dask, Redis, and Ray (cost on log scale).

writing and tuning configurations to align with user intent in diverse and dynamically varying cloud environments. Table IV summarizes the input and output token usage as well as the total cost per configuration for each distributed system, both with and without RAG. Configurations that use RAG incur extremely low costs, typically in the range of a few ten-thousandths of a dollar per run. In addition, hiring skilled DevOps engineers can cost between \$40 and \$100 per hour [53], making automated configuration generation a significantly more economical alternative. Figure 9 compares the cost and time distributions of manual (developer-driven) deployment with those of LADs across Dask, Redis, and Ray. Manual developer time was estimated from developers who conventionally work on distributed systems development, and the corresponding costs were calculated through their publicly available compensations [6], [53]. LADs incurs costs that are four orders of magnitude lower (10^{-4} times cheaper) compared to conventional developers, as shown on the logarithmic scale, and reduces deployment time to nearly zero compared to the 0.5–1.5 hour range typically required by developers. **OB7 By leveraging cost-efficient LLMs with fewer than 20 Billion parameters for frequent API calls, LADs delivers rapid and affordable configuration generation. This makes it a particularly attractive solution for startups that do not have the budget to hire expert DevOps engineers but still want to avail themselves of these high-quality services.**

D. Comparison with recent automated DevOps solutions

We also compare LADs with recent automated DevOps solutions, including IaC-Eval [14] and CloudEval [19]. Both solutions utilize advanced instruction tuning, chain-of-thought alignment, and context-aware techniques. However, they lack the ability to understand system resource boundaries, evolve solutions based on monitoring feedback, and real-world deployment testing. For these reasons, LADs easily outperforms

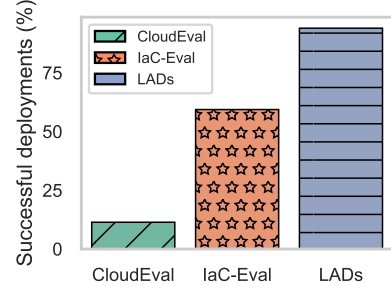


Fig. 10: Comparison with other LLM-based automated DevOps solutions.

both solutions as shown in Figure 10, where CloudEval achieves only 11.4% successful deployments, whereas IaC-Eval achieves a slight improvement of 59.37% due to multiple retries and chain-of-thought reasoning. LADs outperforms them both with a success rate of 94%, highlighting the advantages of using a hybrid mechanism of unifying optimizations into a single system where they are most effective.

VI. LIMITATIONS

Our current evaluation focuses on popular distributed systems that cater to different microservices, however, in the future we plan on extending LADs for unseen systems. This will allow LADs to serve as an automated DevOps solution for developers who create their own custom distributed systems unseen by state-of-the-art LLMs. This will require generating documentation and shots by parsing the code via LLM agents and is part of our future work. However, our current evaluation is focused on seen distributed systems as it is difficult to find unseen distributed systems that are also popular.

VII. CONCLUSION

In this work, we introduced LADs, the first agentic AI framework to provide *end-to-end, runtime-aware automation* of cloud configuration, deployment, and maintenance for complex distributed systems. By tightly integrating instruction prompting (IP), retrieval-augmented generation (RAG), few-shot learning (FSL), chain-of-thought reasoning (CoT), and feedback-based prompt chaining into a closed-loop execution framework, LADs moves beyond prior one-shot LLM-based DevOps tools to continuously diagnose and repair deployment failures. Across 180 deployment tasks spanning Dask, Redis, and Ray, LADs improves deployment success rates from $\leq 22\%$ with simple prompting to 68%, corrects 94% of runtime failures within three iterations, and increases real-world end-to-end deployment success from 11.4% to 94% compared to state-of-the-art automated DevOps approaches. Moreover, LADs generates production-ready configurations in 4–11 seconds at a cost four orders of magnitude lower per invocation than a conventional developer, while substantially reducing manual effort and improving system reliability. Together with our open benchmark and dataset, these results establish LADs as a scalable and cost-efficient foundation for fully automated, error-resilient DevOps in cloud-native environments.

REFERENCES

- [1] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica *et al.*, “A view of cloud computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [2] F. N. A. Sackey, “Strategies to manage cloud computing operational costs,” Ph.D. dissertation, Walden University, 2018.
- [3] R. R. Dhal and B. P. Rao, “Service cost estimation in cloud environment using a third party web server: A comparative analysis with and without using cloud computing,” *IOSR Journal of Computer Engineering (IOSR-JCE)*, e-ISSN, pp. 2278–0661, 2018.
- [4] X. Lian *et al.*, “Configuration validation with large language models,” *University of Illinois at Urbana-Champaign*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.09690v2>
- [5] X. Lian, “Configuration validation with large language models,” *Arxiv*, 2024. [Online]. Available: <https://arxiv.org/pdf/2310.09690>
- [6] C. Slingerland, “55 fascinating DevOps statistics you need to know in 2024,” <https://www.cloudzero.com/blog/devops-statistics/>, 2024, accessed: 2024-11-18.
- [7] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, “Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications,” *ACM Transactions on Storage (TOS)*, vol. 17, no. 4, pp. 1–32, 2021.
- [8] J. Lao, Y. Wang, Y. Li, J. Wang, Y. Zhang, Z. Cheng, W. Chen, M. Tang, and J. Wang, “Gptuner: A manual-reading database tuning system via gpt-guided bayesian optimization,” *Proceedings of the VLDB Endowment*, vol. 17, no. 8, pp. 1939–1952, 2024.
- [9] V. Thakkar, M. Sukumar, J. Dai, K. Singh, and Z. Cao, “Can modern llms tune and configure lsm-based key-value stores?” in *Proceedings of the 16th ACM Workshop on Hot Topics in Storage and File Systems*, 2024, pp. 116–123.
- [10] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A {fault-tolerant} abstraction for {in-memory} cluster computing,” in *9th USENIX symposium on networked systems design and implementation (NSDI 12)*, 2012, pp. 15–28.
- [11] C. Zheng, Q. Zhuang, and F. Guo, “A multi-tenant framework for cloud container services,” in *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2021, pp. 359–369.
- [12] Y. Zhang, J. Yang, Z. Jin, U. Sethi, K. Rodrigues, S. Lu, and D. Yuan, “Understanding and detecting software upgrade failures in distributed systems,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021, pp. 116–131.
- [13] B. Rabenstein and J. Volz, “Prometheus: A next-generation monitoring system (talk).” Dublin: USENIX Association, May 2015.
- [14] P. T. J. Kon, J. Liu, Y. Qiu, W. Fan, T. He, L. Lin, H. Zhang, O. Park, G. Elengikal, Y. Kang, A. Chen, M. Chowdhury, M. Lee, and X. Wang, “IaC-Eval: A code generation benchmark for Infrastructure-as-Code programs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, December 2024, poster presentation. [Online]. Available: <https://neurips.cc/virtual/2024/poster/97835>
- [15] F. Li, H. Lang, J. Zhang, J. Shen, and X. Wang, “PreConfig: A pretrained model for automating network configuration,” *arXiv preprint arXiv:2403.09369*, 2024.
- [16] Y. Fu, “MissConf: LLM-Enhanced reproduction of configuration-triggered bugs,” in *ICSE-Companion*, 2024. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3639478.3647635>
- [17] J. Kannan, “Using language models for software tool configuration,” *Arxiv*, 2023. [Online]. Available: <https://arxiv.org/pdf/2312.06121v1>
- [18] R. M. *et al.*, “What do LLMs need to synthesize correct router configurations?” in *HotNets*, 2023.
- [19] Y. Xu, Y. Chen, X. Zhang, X. Lin, P. Hu, Y. Ma, S. Lu, W. Du, Z. Mao, E. Zhai, and D. Cai, “Cloudeval-yaml: A practical benchmark for cloud configuration generation,” in *Proceedings of the 7th Conference on Machine Learning and Systems (MLSys)*, 2024.
- [20] Y. Chen, M. Shetty, G. Somashekar, M. Ma, Y. Simmhan, J. Mace, C. Bansal, R. Wang, and S. Rajmohan, “Aiopslab: A holistic framework for evaluating ai agents for enabling autonomous cloud,” in *MLSys ’25*, May 2025. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/aiopslab-a-holistic-framework-for-evaluating-ai-agents-for-enabling-autonomous-cloud/>
- [21] F. V. Ruiz, “Agent-driven automatic software improvement,” *International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.16739>
- [22] E. Malul, “GenKubeSec: LLM-Based kubernetes misconfiguration detection, localization, reasoning, and remediation,” *Arxiv*, 2024. [Online]. Available: <https://arxiv.org/pdf/2405.19954>
- [23] D. D. Team, “Dask: Library for dynamic task scheduling,” 2016. [Online]. Available: <https://dask.org>
- [24] R. Ltd., “Redis: In-memory data structure store,” 2009. [Online]. Available: <https://redis.io>
- [25] R. Nishihara, P. Moritz, and contributors, “Ray: A unified framework for scalable and distributed ML,” <https://www.ray.io/>, 2017, distributed computing framework for machine learning and Python applications.
- [26] L. Zhong, Z. Wang, and J. Shang, “Ldb: A large language model debugger via verifying runtime execution step-by-step,” *arXiv preprint arXiv:2402.16906*, 2024.
- [27] Y. Hu, Q. Zhou, Q. Chen, X. Li, L. Liu, D. Zhang, A. Kachroo, T. Oz, and O. Tripp, “QualityFlow: An agentic workflow for program synthesis controlled by LLM quality checks,” *arXiv preprint arXiv:2501.17167*, 2025.
- [28] D. Huang, Q. Bu, J. M. Zhang, M. Luck, and H. Cui, “Agentcoder: Multi-agent-based code generation with iterative testing and optimisation,” *arXiv preprint arXiv:2312.13010*, 2023.
- [29] Z. Liang, Y. Liu, T. Niu, X. Zhang, Y. Zhou, and S. Yavuz, “Improving llm reasoning through scaling inference computation with collaborative verification,” *arXiv preprint arXiv:2410.05318*, 2024.
- [30] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, “An empirical study of the non-determinism of ChatGPT in code generation,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, Jan. 2025. [Online]. Available: <https://doi.org/10.1145/3697010>
- [31] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. Dai, and Q. Le, “Finetuned language models are zero-shot learners,” *arXiv preprint arXiv:2109.01652*, 2021.
- [32] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [33] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [34] T. Gao, A. Fisch, and D. Chen, “Making pre-trained language models better few-shot learners,” in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021, pp. 3816–3830.
- [35] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *arXiv preprint arXiv:2201.11903*, 2022.
- [36] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [37] F. Cuconasu, G. Trappolini, F. Siciliano, S. Filice, C. Campagnano, Y. Maarek, N. Tonello, and F. Silvestri, “The power of noise: Redefining retrieval for RAG systems,” in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 719–729. [Online]. Available: <https://doi.org/10.1145/3626772.3657834>
- [38] n8n.io, *n8n: Free and open fair-code workflow automation tool*, n8n, 2026, software available from <https://n8n.io>. [Online]. Available: <https://n8n.io>
- [39] Amazon Web Services, *Amazon Elastic Compute Cloud (EC2)*, Amazon Web Services, Inc., 2026, available at <https://aws.amazon.com/ec2/>.
- [40] —, *Amazon Simple Storage Service (S3)*, Amazon Web Services, Inc., 2026, software available from <https://aws.amazon.com/s3/>.
- [41] O. Chebaro, N. Kosmatov, A. Giorgetti, and J. Julliand, “Program slicing enhances a verification technique combining static and dynamic analysis,” in *Proceedings of the 27th Annual ACM Symposium on Applied Computing*, ser. SAC ’12. New York, NY, USA: Association for Computing Machinery, 2012, p. 1284–1291. [Online]. Available: <https://doi.org/10.1145/2245276.2231980>
- [42] Dask Development Team, “Scaling and performance with dask,” 2017, accessed: 2025-02-13. [Online]. Available: <https://blog.dask.org/2017/07/03/scaling>

- [43] Redis, “Redis benchmarks - performance optimization,” 2025, accessed: 2025-02-13. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/
- [44] Ray Project, “KubeRay: Deploy and manage ray on kubernetes,” 2025, accessed: 2025-02-13. [Online]. Available: <https://github.com/ray-project/kuberay>
- [45] W. John, G. Marchetto, F. Nemeth, P. Skoldstrom, R. Steinert, C. Meirosu, I. Papafili, and K. Pentikousis, “Service provider DevOps,” *IEEE Communications Magazine*, vol. 55, no. 1, pp. 204–211, 2017.
- [46] Ollama, “Qwen2.5-Coder: AI model for code generation,” accessed: February 2025. [Online]. Available: <https://ollama.com/library/qwen2.5-coder>
- [47] DeepSeek, “DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence,” June 2024, accessed: February 2025. [Online]. Available: <https://ollama.com/library/deepseek-coder-v2>
- [48] Ollama, “CodeLlama: AI model for code generation,” accessed: February 2025. [Online]. Available: <https://ollama.com/library/codellama>
- [49] —, “Llama 3.1: Advanced AI language model,” accessed: February 2025. [Online]. Available: <https://ollama.com/library/llama3.1>
- [50] Hugging Face, “Hugging face: The AI community building the future,” accessed: February 2025. [Online]. Available: <https://huggingface.co>
- [51] Ollama, “Ollama: Run AI models locally,” accessed: February 2025. [Online]. Available: <https://ollama.com>
- [52] S. Qiao, R. Fang, Z. Qiu, X. Wang, N. Zhang, Y. Jiang, P. Xie, F. Huang, and H. Chen, “Benchmarking agentic workflow generation,” *arXiv preprint arXiv:2410.07869*, 2024.
- [53] Upwork, “How much does it cost to hire a DevOps engineer?” 2025, accessed: 2025-02-15. [Online]. Available: <https://www.upwork.com/hire/devops-engineers/cost/>